

A Radix-4 Booth Multiplier in Structural VHDL

Architecture, optimization and synthesis at 32 bits in Nangate 45 nm

Erfan Taheri

Synopsys Design Compiler and ModelSim

Project scope and result

Engineering question

How close can a hand-built signed multiplier come to the implementation that Design Compiler infers from $A * B$ under the same library and constraints?

Implementation

A configurable structural VHDL datapath with a radix-4 Booth front end, a Wallace or Dadda reduction tree, and a sparse carry-propagate adder.

Three changes

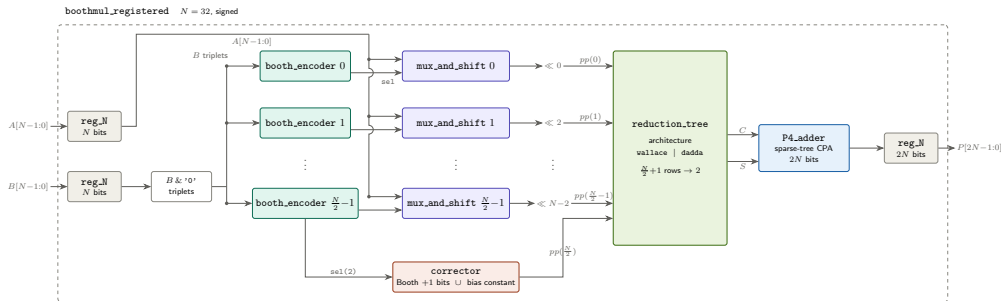
Remove explicit sign extension, generate a Dadda schedule at elaboration, then merge Booth selection with conditional negation.

Final architecture across the sweep

Fastest derived period	1.75 ns
Reference	1.63 ns
Smallest area	5029 μm^2
Reference	4441 μm^2
Area gap recovered	69.1%

At the common 3.0 ns target, the final architecture is 21.2% smaller than the structural baseline and uses 44.1% less dynamic power than plain Dadda.

Multiplier architecture



Only the row generator, corrector and reduction-tree architecture change between structural variants.
VHDL configurations select each combination.

Radix-4 Booth recoding

Overlapping triplets of the multiplier operand select one signed digit $d_i \in \{0, \pm 1, \pm 2\}$:

$$AB = \sum_{i=0}^{N/2-1} d_i A 4^i, \quad B_{-1} = 0$$

triplet	d_i	generated value
000, 111	0	0
001, 010	+1	+A
011	+2	+2A
100	-2	-2A
101, 110	-1	-A

The recoder reduces the partial-product count from N to $N/2$. At $N = 32$, the datapath reduces 16 Booth rows plus one correction row.

The shift for $2A$ changes wiring positions. Conditional negation adds logic to every row and becomes an important optimization target.

Sign-extension elimination

Each Booth value fits in $N + 1$ two's-complement bits. Explicit sign extension adds $N^2/4 = 256$ replicated bits to the 32-bit partial-product array.

For sign bit s , the identity $-s = \bar{s} - 1$ gives

$$-s2^N = \bar{s}2^N - 2^N.$$

The row keeps its lower N bits, stores the inverted sign bit, and fills the upper positions with zero. Row i then has a fixed positive bias 2^{N+2i} , independent of the operands.

$$C = - \sum_{i=0}^{N/2-1} 2^{N+2i} \pmod{2^{2N}} = 0xAAAAAAAAAB \ll N \quad (N = 32)$$

The correction constant occupies the upper half of the existing Booth increment row. It needs no additional reduction-tree row.

Array dots: 800 to 561

Minimum area: 6346 to 5975 μm^2

Generated Dadda reduction

The Wallace baseline instantiates full-width carry-save adders. The Dadda architecture instead places compressors only where a column exceeds the next target height.

$$d_0 = 2, \quad d_{j+1} = \left\lfloor \frac{3}{2} d_j \right\rfloor \quad 2, 3, 4, 6, 9, 13, 19, \dots$$

At $N = 32$, the elaboration-time schedule uses six stages with targets 13, 9, 6, 4, 3, 2.

The scheduler must include carries arriving from column $k - 1$:

$$h_{\text{next}} = C_{\text{in}} + n - 2F - H \leq d_{\text{target}}.$$

Wallace RTL instances	960 FA
Dadda schedule	435 FA, 45 HA
Minimum area	5975 to 5620 μm^2
Fastest T_0	1.87 to 1.73 ns

It records full-adder sources, half-adder sources and pass-through dots in three matrices. Generate statements use those matrices for exact port maps.

Fused Booth selector

The original row generator first selected A , $2A$ or zero, then applied an explicit conditional inversion to every bit. The revised architecture exposes selection and negation as one Boolean function per bit.

$$q_i = c_{+1}a_i + c_{-1}\bar{a}_i + c_{+2}(2a)_i + c_{-2}\overline{(2a)_i}.$$

The four controls are mutually exclusive. A zero row sets all four controls to zero. Design Compiler can therefore map the combined cone without preserving a separate XOR boundary.

at a 3.0 ns constraint	XOR/XNOR	FA_X1	cells	area
Dadda, separate negation	1230	117	3936	5725 μm^2
Dadda, fused selector	85	448	2819	5125 μm^2
Inferred $A * B$	559	268	3035	4910 μm^2

The paired shift in XOR and adder cells supports improved adder recognition, while the total cell count falls by 28.4%.

Verification and measurement

Functional checks

width	test coverage
$N = 8$	all 65 536 input pairs
$N = 32$	corner cases and 20 000 random pairs

One self-checking testbench compares every variant with `signed(A) * signed(B)`. Matching VHDL configurations bind the same architectures for simulation and synthesis.

The power flow uses maximum-delay SDF activity from the mapped netlists and reaches 100% SAIF annotation in every committed report.

Synthesis setup

Library	Nangate 45 nm, typical ECSM
Tool	DC W-2024.09-SP2
Flow	<code>compile_ultra</code>
Area/timing	21 common, 12 extra Dadda runs
Power	120 activity-annotated points

The setup uses registered inputs and outputs, fixed I/O delays and the same script for every variant.

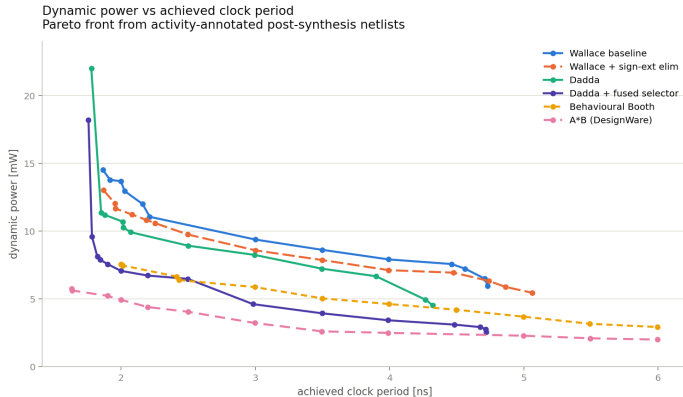
Area at a common 3.0 ns constraint

variant	cell area	total cells	area change from prior
Wallace baseline	6501 μm^2	4639	baseline
Sign extension removed	6196 μm^2	4455	−4.7%
Dadda reduction	5725 μm^2	3936	−7.6%
Fused selector	5125 μm^2	2819	−10.5%
Inferred A * B	4910 μm^2	3035	reference

The three changes reduce area by **21.2%** relative to the structural baseline at the same clock constraint.

At this point, the final design is **4.4%** larger than the inferred reference and uses fewer total cells. The cell mix explains why count and area do not rank identically.

Post-synthesis dynamic power



At the 3.0 ns synthesis target

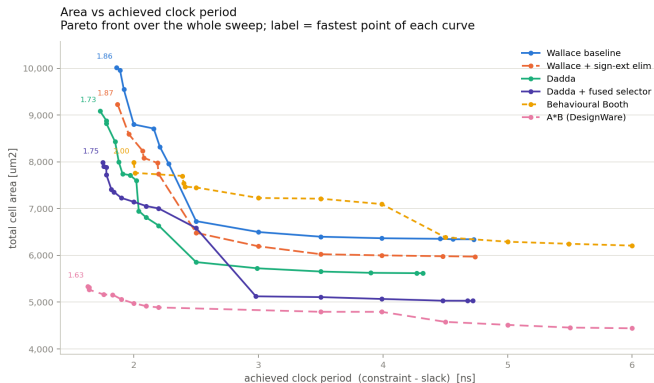
Wallace baseline	9.38 mW
Plain Dadda	8.25 mW
Fused Dadda	4.61 mW
Inferred A * B	3.22 mW

The fused selector uses **44.1% less dynamic power** than plain Dadda at this point.

Each netlist ran 500 vectors with maximum-delay SDF at its achieved period. All reports show 100% SAIF annotation.

Power uses 20 common targets from 1.0 to 6.0 ns and its own Pareto front.

Area and timing trade-off



For each synthesized netlist, $T_0 = T_{\text{constraint}} - \text{slack}$ gives the clock period at zero setup slack under the fixed constraints. The plot keeps only the non-dominated points from the sweep.

The next step is placement, clock-tree synthesis and routing.

Timing and power can then be measured again with extracted parasitics to obtain more precise final implementation results.